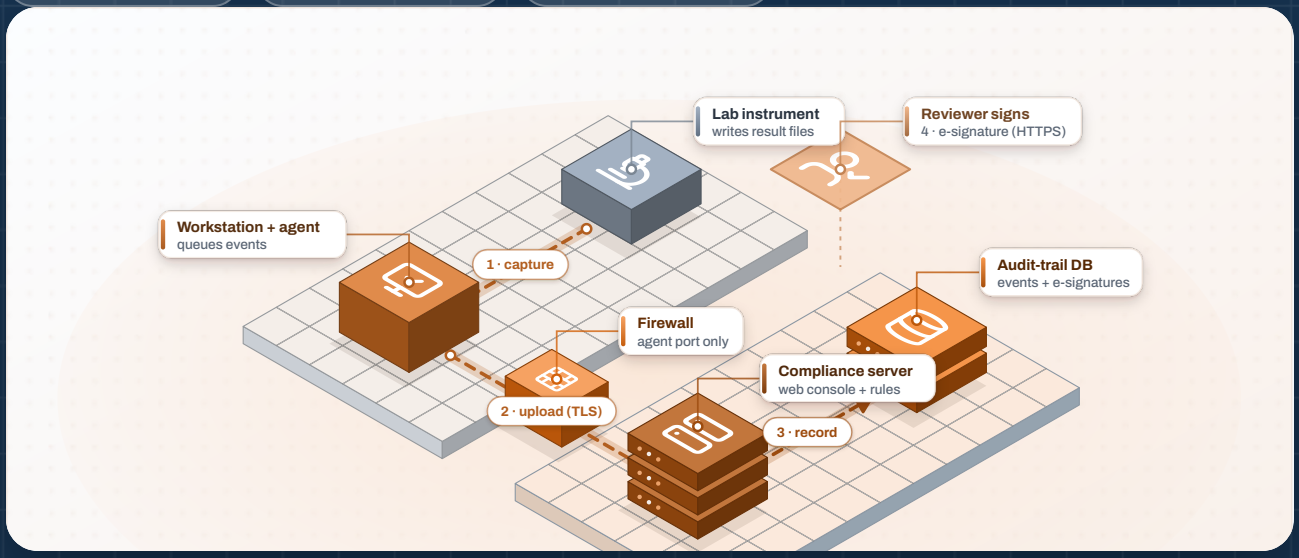
STANDARD OPERATING PROCEDURE · FIELD GUIDE SERIES

SOP-LAB-002

Installing, Registering and Patching a Lab Compliance Data Agent

A Compliance Builder example: register, install, connect and hotfix a GxP data-capture agent four ways (interactive, silent msiexec + PowerShell, remote via BeyondTrust PRA or PowerShell remoting, and ConfigMgr / Intune Win32 deployment).

Regulated Lab IT | GxP Data Integrity | Endpoint Software



DOCUMENT ID	SOP-LAB-002
VERSION	2.0
EFFECTIVE	October 2026
AUDIENCE	Lab IT specialists, desktop engineers, CSV / quality partners
AUTHOR	Richmond Dieujuste, Principal Consultant

Rich Dieu IT Solutions LLC · richdieu.com

Sanitized reference edition — client identifiers removed

Contents

1. Concepts You Need First 3

1.1 Agent and server architecture 3

1.2 Why a workstation must be registered first 4

1.3 Service accounts 4

1.4 Ports and firewall 5

1.5 Hotfixes in a validated environment 5

1.6 GxP evidence in one paragraph 5

1.7 Choosing a method 6

2. Access, Prerequisites and Planning 8

2.1 Level of access required 8

2.2 Prerequisites checklist 9

2.3 Pre-flight script 9

2.4 Decisions to make up front 9

2.5 Preparing the server side (web console) 10

3. Methods A and B · Interactive and Silent Installation 11

3.1 Method A · Interactive installation 11

3.2 Method B · Silent install with msiexec and PowerShell 12

4. Methods C and D · Remote and Enterprise Deployment 16

4.1 Method C · Remote installation 16

4.2 Method D · Enterprise deployment (ConfigMgr or Intune) 18

5. Hotfix, Verification and Evidence 20

5.1 The hotfix lifecycle 20

5.2 Verification checklist 21

5.3 Evidence to capture 21

6. Rollback and Troubleshooting 23

6.1 Rollback 23

6.2 Troubleshooting flow 23

7. Summary, Glossary and Review 25

7.1 Chapter summary 25

7.2 Glossary 25

LEARNING OBJECTIVES

After working through this procedure you will be able to:

- Explain how an agent/server compliance data-capture system works and why it matters for 21 CFR Part 11 and EU Annex 11.
- Plan the service account, network ports and firewall rules an agent needs before touching the workstation.
- Register a workstation in the web console, then install and connect the agent interactively or silently with msiexec and PowerShell.
- Deploy the agent remotely (BeyondTrust PRA, PowerShell remoting) and at scale (ConfigMgr application or Intune Win32 app).
- Apply a vendor hotfix under change control, with a tested rollback path.
- Capture GxP-quality evidence (versions, checksums, screenshots, time sync) for the change record.

TIME	DIFFICULTY	MINIMUM ACCESS	RISK
≈ 30–45 min per workstation	Intermediate	Local admin + console workstation-admin role	Medium (validated system; change control)

CHAPTER 1

Concepts You Need First

In a regulated laboratory, the number a balance prints or the chromatogram an HPLC writes to disk is not just data. It is a *record* that may end up supporting a product release, a regulatory submission or a patient-safety decision. Regulators therefore expect the lab to prove that every such record is complete, unaltered, attributable to a person and retained for its full life. A **compliance data-capture system** is the software that provides that proof. This SOP uses Compliance Builder as its worked example, but the same pattern applies to any product that pairs a **workstation agent** with a **central server**.

> **KEY CONCEPT · THE AGENT IS A NOTARY AT THE INSTRUMENT**
>
> Picture a notary sitting beside each instrument PC. Whenever a result file is created, changed, renamed or deleted, the notary writes down what happened, when, and who was signed in, then posts a sealed copy to the central records office. The analyst can later countersign the entry. The notary is the *agent*; the records office is the *server and its database*; the countersignature is an *electronic signature*. If the notary is missing, asleep or cannot reach the office, records pile up unattested, and that is exactly the failure this SOP is designed to prevent.

1.1 Agent and server architecture

A typical deployment has three tiers:

- **Agent**: a Windows service on each instrument workstation. It watches configured folders or applications, records *events* (file created, modified, deleted, printed), queues them locally, and transmits them to the server. A desktop component, the agent window, lets analysts review events and sign them off with their domain credentials.
- **Server / web console**: receives events, applies the rules of the *System* the workstation belongs to (which folders, which file types, who may sign), and gives administrators a browser console to register workstations and view systems. In this SOP the console is called the *web console*.
- **Database**: the durable, access-controlled audit trail. Records here are what an inspector will ask to see.

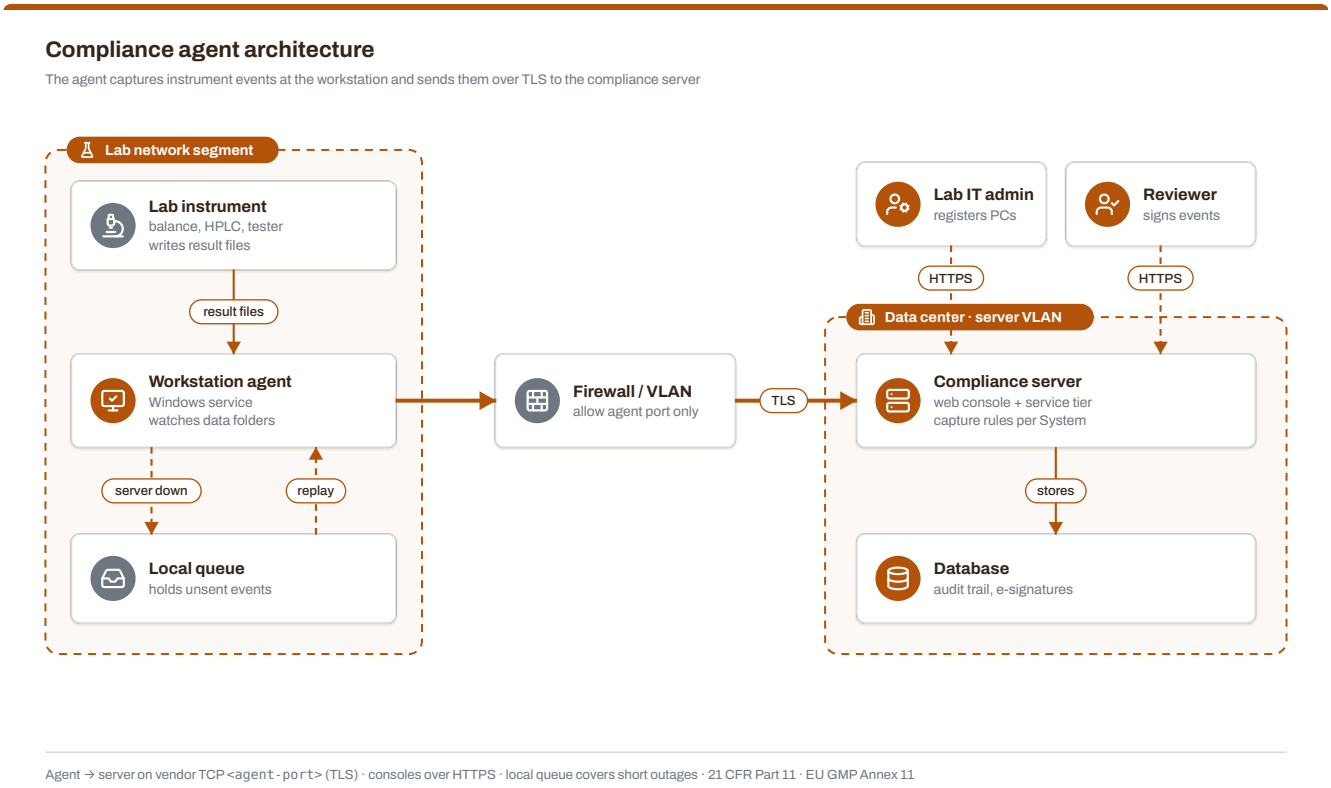


Figure 1. The local queue is what lets a lab survive a short network outage without losing records, so a growing backlog is an incident to investigate, not noise.

1.2 Why a workstation must be registered first

An agent that connects to a server the server has never heard of is, from a data-integrity standpoint, an unknown device. Most products therefore refuse or quarantine connections from unregistered workstations. Registration in the web console ties the workstation's hostname to a **System** (for example "Mechanical Test Lab · Tensile testers"), and the System supplies the capture rules. That is why the source procedure starts in the console, not on the PC, and why some environments need a *System Reload* after a new workstation is added: the server must reload its configuration before it will accept the agent.

1.3 Service accounts

The agent service runs whether or not anyone is signed in, so it needs an identity of its own. The options, from most to least preferred:

Identity	When to use	Watch out for
Vendor-default (often LocalSystem)	Agent only reads local folders and talks outbound to the server.	Very powerful locally; cannot reach network shares with its own rights.

Group managed service account (gMSA)	Agent must read or write network shares; vendor supports gMSA.	Not every vendor supports it. Confirm in vendor documentation before relying on it.
Domain service account	Network access needed and gMSA unsupported.	Password rotation must be planned; grant *Log on as a service* only, deny interactive logon.

WHY IT MATTERS · ATTRIBUTABILITY

Regulations require every action to be attributable. If the service account is shared with other tools, or if analysts know its password, the audit trail can no longer prove who did what. Keep the service account dedicated, non-interactive and owned by IT.

1.4 Ports and firewall

The agent talks to the server over a TCP port defined by the vendor and by your installation (shown in this SOP as `<agent-port>`). Some products also have the server call back to the agent, which requires an *inbound* rule on the workstation. Lab PCs often sit on a segmented instrument VLAN with restrictive ACLs, so the network path must be requested before installation day. Confirm three things with the vendor documentation and the server owner: the port number(s), the direction of each connection, and whether traffic is TLS-encrypted.

1.5 Hotfixes in a validated environment

A hotfix (in the reference engagement, the vendor's "CB10" hotfix) changes validated software. In GxP terms that is a *change to a computerized system*, so it follows a lifecycle: impact assessment, testing on a non-production system, approval, deployment in a maintenance window, verification and documented closure. Chapter 5 walks through it.

1.6 GxP evidence in one paragraph

US 21 CFR Part 11 and EU GMP Annex 11 require that electronic records are trustworthy and that changes to the systems that create them are controlled. The working shorthand is **ALCOA+**: records are Attributable, Legible, Contemporaneous, Original and Accurate, plus Complete, Consistent, Enduring and Available. Every evidence item this SOP asks you to capture (versions, checksums, timestamps, screenshots, operator identity) exists to support one of those letters.

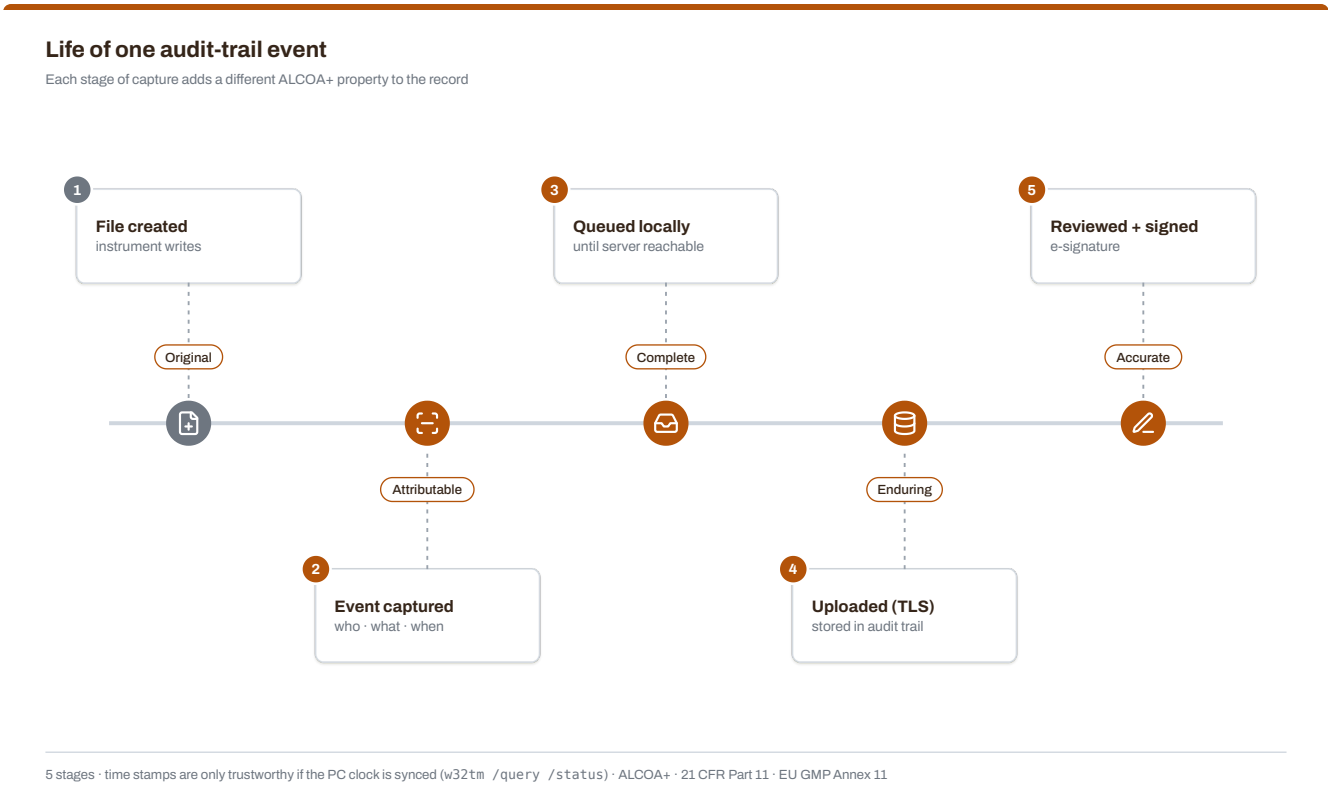


Figure 2. A broken agent, a deleted queue or an unsynchronized clock each removes an ALCOA+ property from the record, which makes them data-integrity problems rather than ordinary IT faults.

1.7 Choosing a method

A · Interactive install

Run the MSI as administrator at the console, configure the connection in the Agent Configuration Utility.

BEST FOR: FIRST INSTALL, PILOT, LEARNING

B · Silent msiexec + PowerShell

Unattended install with logging, service hardening and an evidence file.

BEST FOR: REPEATABLE BATCHES

C · Remote (PRA / WinRM)

BeyondTrust PRA session or PowerShell remoting from a jump host.

BEST FOR: REMOTE SUPPORT, LOCKED-DOWN LABS

D · ConfigMgr / Intune

Package once as an application or Win32 app; target a device collection or group.

BEST FOR: FLEETS, HOTFIX ROLL-OUTS

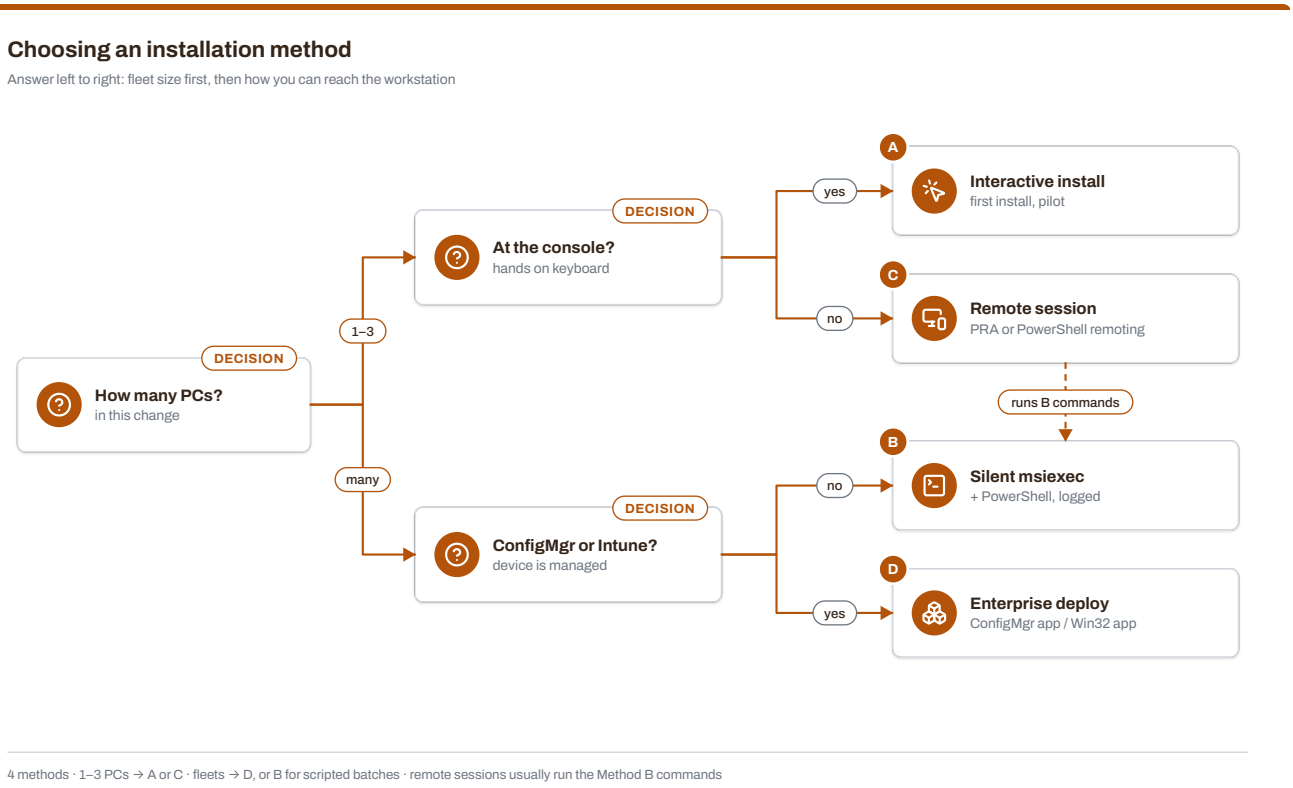


Figure 3. Fleet size decides the method more than preference does, and remote sessions usually end up running the Method B commands anyway.

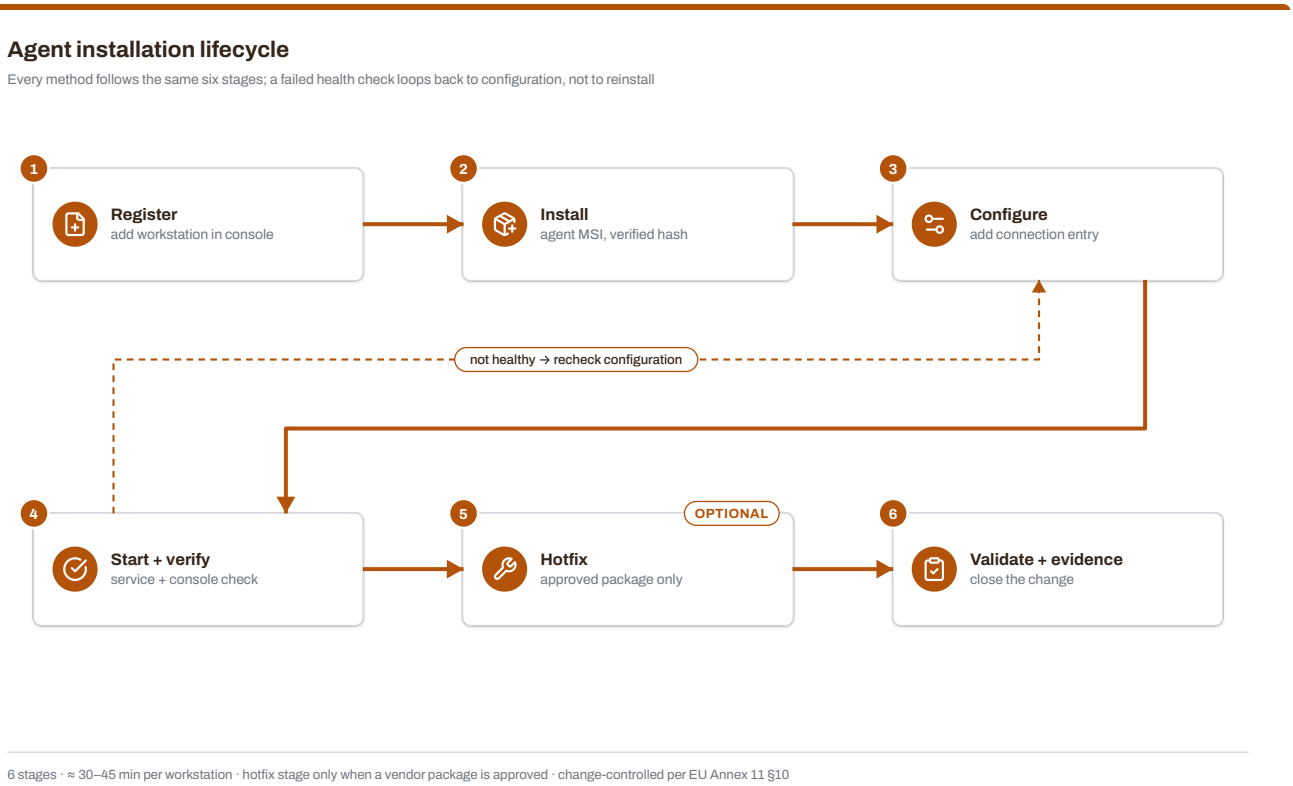


Figure 4. When verification fails, go back to the connection entry; reinstalling first only hides the real cause.

CHAPTER 2

Access, Prerequisites and Planning

2.1 Level of access required

Task	Least-privilege access	Notes
Register workstation, System Reload	Web console role that can add Workstations (application administrator or equivalent)	Role names are vendor- and site-specific. Viewing Systems usually needs only a read role.
Install / uninstall the agent and hotfix	Local Administrators group on the workstation	Use a separate admin account (or LAPS-managed local admin), never the analyst's account.
Set the service logon account	Local admin + the service account's credentials, or a gMSA the computer is allowed to retrieve	Account needs *Log on as a service*; deny interactive and RDP logon via GPO.
Firewall / ACL change	Network or firewall team via change request	Lab VLAN ACLs are rarely under desktop IT control.
Remote session (Method C)	BeyondTrust PRA user with access to the Jump Group holding the workstation; or membership in the workstation's `Remote Management Users` / Administrators for WinRM	PRA policies can require approval and session recording, which double as evidence.
ConfigMgr deployment (Method D)	ConfigMgr *Application Author* to create, *Application Deployment Manager* to deploy (scoped to the lab collection)	Use security scopes so lab apps cannot be deployed to the wider estate by mistake.
Intune deployment (Method D)	Intune *Application Manager* role (scope tag: Lab)	Entra ID role *Intune Administrator* also works but is far broader than needed.

GOOD PRACTICE · JUST-IN-TIME ADMIN

Where available, use Privileged Identity Management or a privileged access workstation to activate admin rights only for the change window. The activation log itself becomes evidence that elevated access was approved and time-bound.

2.2 Prerequisites checklist

- [ ] Approved change record (for example `CHG0012345`) and maintenance window agreed with the lab, because instrument runs must be paused.
- [ ] Approved agent installer (`CBAgentSetup.msi` in the reference engagement) and hotfix package from the controlled software repository, with vendor checksums.
- [ ] Workstation hostname and OS version confirmed (Windows 10 or 11, ideally an LTSC edition for lab PCs).
- [ ] Server address (`<cb-server.contoso.com>`), port (`<agent-port>`) and authentication method for the connection entry.
- [ ] Service account or gMSA created, if the System requires network access.
- [ ] Firewall path open and tested (pre-flight script below).
- [ ] System time synchronized with the domain (`w32tm /query /status`), because audit-trail timestamps depend on it.
- [ ] Remote access arranged if working off-site (BeyondTrust PRA invite or Jump Client).

2.3 Pre-flight script

Run this on the workstation (or through remoting) before the window. It answers the questions that most often derail an installation.

TEST-AGENTPREREQS.PS1 POWERSHELL

```powershell
param(
  [Parameter(Mandatory)] [string] $Server,          # e.g. cb-server.contoso.com
  [Parameter(Mandatory)] [int]    $Port             # vendor agent port
)
$os = Get-CimInstance Win32_OperatingSystem
[pscustomobject]@{
  Hostname     = $env:COMPUTERNAME
  OS           = "$($os.Caption) $($os.Version)"
  IsAdmin      = ([Security.Principal.WindowsPrincipal]
[Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole('Administrators')
  TimeSource   = (w32tm /query /source).Trim()
  DnsResolves  = [bool](Resolve-DnsName $Server -ErrorAction SilentlyContinue)
  PortOpen     = (Test-NetConnection $Server -Port $Port -WarningAction
SilentlyContinue).TcpTestSucceeded
  FreeGB_C     = [math]::Round((Get-PSDrive C).Free/1GB,1)
  PendingReboot= Test-Path 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Component Based
Servicing\RebootPending'
} | Format-List
```

2.4 Decisions to make up front

Decision	Guidance
Which System	Confirm with the lab and the system owner which System the workstation joins; wrong placement means wrong capture rules.

Service identity	Vendor default unless network shares are monitored; then gMSA or dedicated domain account.
Install and hotfix together?	Yes for new workstations if the hotfix is already approved: install base, verify, then hotfix in the same window. Record both versions.
Method	See the decision tree; single PCs with a technician on site use A, everything else B, C or D.
Re-validation	Ask Quality whether the hotfix needs executed OQ test scripts on each workstation or only on a representative one.

2.5 Preparing the server side (web console)

Every method starts here. Doing the console work first means the agent can connect the moment it is installed, and it gives you a place to see the workstation appear.

1. Sign in to the web console with your administrator role.
2. Open **Systems** and locate the target System. Record its name and the server/port its agents should use.
3. Open **Workstations** and select **Add**. Enter the hostname exactly as Windows reports it (`hostname` or `$env:COMPUTERNAME`) and the required metadata: System, location (lab, not building or room in public documents), owner, purpose.
4. Save. If the environment requires it, perform a **System Reload** so the server picks up the new workstation and any configuration changes.
5. Take a screenshot of the new workstation entry for the change record.

WATCH OUT · HOSTNAME MISMATCHES

Registration is keyed on the hostname. A typo, an old name after a re-image, or a NetBIOS name that differs from the DNS name is the single most common reason a correctly installed agent never shows as connected.

CHAPTER 3

Methods A and B · Interactive and Silent Installation

3.1 Method A · Interactive installation

The interactive method is how you learn the product and how you perform the first installation in a pilot. It is also the method the source procedure documents.

METHOD A **Install and configure at the console**

1. Sign in with your admin account (on site, or through a remote session as in Method C). Confirm the hostname and OS: Win+R, `winver`, and `hostname` in a command prompt.
2. Copy the approved `CBAgentSetup.msi` from the software repository to `C:\Install\CBAgent\`. Verify its hash against the repository: `Get-FileHash .\CBAgentSetup.msi -Algorithm SHA256`.
3. Right-click the MSI and choose **Install**, or launch an elevated command prompt and run `msiexec /i CBAgentSetup.msi` so you get UAC elevation reliably.
4. Complete the wizard per site standard: accept the default install folder unless your build standard says otherwise, and supply the service account if prompted.
5. Open the **Agent Configuration Utility** from the Start menu.
6. Go to **Connections** and add an entry for the correct System: server address, port and authentication as required. Save and confirm the entry is listed.
7. Open **services.msc**, find the agent service, confirm *Startup type: Automatic*, and **Start** or **Restart** it.
8. Open the agent window and check the **Connections** tab for a healthy state. In the web console, confirm the workstation now reports under **Workstations** and is tied to the intended System.

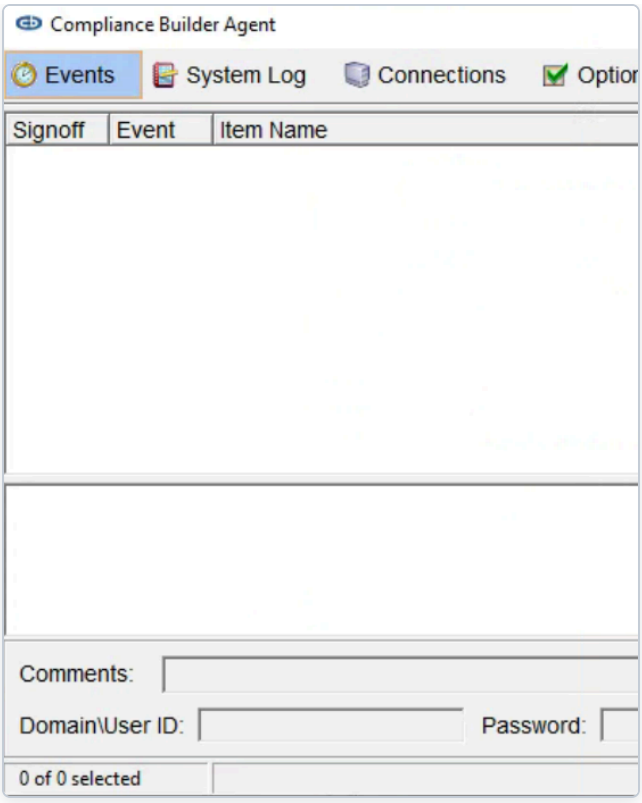


Figure 5. The agent window on a lab workstation. Events are listed with their sign-off state; the Comments and Domain\User ID / Password fields are used to sign events electronically. The Connections tab shows the link to the server.

> **NOTE · IF THE STATE STAYS DISCONNECTED**
>
> When the configuration is correct but the connection does not come up, a single clean reboot often resolves it, because some agents only bind their network listener at boot. In the reference engagement one hard reset brought the connection up once the connection entry was correct. Do one reboot, not several, and record it.

3.2 Method B · Silent install with msiexec and PowerShell

Windows Installer packages (MSI) support an unattended mode. Scripting the install removes click-by-click variation between technicians, produces a verbose log, and lets you harden the service and write evidence in the same run. This is the core that Methods C and D reuse.

The msiexec command line

SILENT INSTALL (ELEVATED) CMD

```
msiexec /i "C:\Install\CBAgent\CBAgentSetup.msi" /qn /norestart /l*v
"C:\Windows\Temp\CBAgent_install.log" ^
  <SERVERPROPERTY>="cb-server.contoso.com" <PORTPROPERTY>="<agent-port>"
```

Switch	Meaning
`/i`	Install the package.
`/qn`	No user interface at all (use `/qb` for a basic progress bar while testing).
`/norestart`	Never reboot automatically; you decide when, inside the window.
`/l*v`	Verbose log of everything. Keep it as evidence.
`PROPERTY=value`	Public MSI properties. Whether the vendor exposes server and port as properties, and their names, is product-specific. If not, configure the connection afterwards (Agent Configuration Utility or a vendor-supplied config file).

GOOD PRACTICE · FIND THE PROPERTIES YOURSELF

Open the MSI in Orca (Windows SDK) or run an administrative install (`msiexec /a`) and read the *Property* table. Upper-case property names are public and can be set on the command line. Exit code `0` is success and `3010` is success with reboot required; anything else, read the log from the bottom up for "Return value 3".

Base template

INSTALL-COMPLIANCEAGENT.PS1 · BASE TEMPLATE POWERSHELL

```powershell
<#
.SYNOPSIS  Install (or hotfix) a compliance data-capture agent silently, harden its service, write
evidence.
.NOTES     Run elevated on the workstation (or via Invoke-Command). Vendor-specific values are
<placeholders>.
#>
[CmdletBinding(SupportsShouldProcess)]
param(
  [Parameter(Mandatory)] [string] $MsiPath,                  # \\repo\...\CBAgentSetup.msi or local
copy
  [Parameter(Mandatory)] [string] $ExpectedSha256,           # from the controlled repository
  [Parameter(Mandatory)] [string] $ChangeRef,                # e.g. CHG0012345
  [string]   $ServiceName   = '<AgentServiceName>',          # confirm with: Get-Service *agent*
  [string]   $MsiProperties = '',                            # e.g. 'SERVER=cb-server.contoso.com PORT=
<agent-port>'
  [string]   $HotfixPath    = '',                            # optional hotfix installer (.msp or .msi)
  [pscredential] $ServiceCredential,                         # optional domain service account
  [string]   $EvidenceDir   = 'C:\ProgramData\LabIT\Evidence'
)
$ErrorActionPreference = 'Stop'
New-Item $EvidenceDir -ItemType Directory -Force | Out-Null
$stamp = Get-Date -Format 'yyyyMMdd-HHmmss'
$ev    = [ordered]@{ ChangeRef=$ChangeRef; Host=$env:COMPUTERNAME;
Operator="$env:USERDOMAIN\$env:USERNAME"; StartUtc=(Get-Date).ToUniversalTime() }

function Get-AgentVersion {
  Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*',
                   'HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*' -ErrorAction
SilentlyContinue |
    Where-Object DisplayName -like '*Compliance Builder*' | Select-Object -First 1 DisplayName,
```

```
DisplayVersion, PSChildName
}

# ---- Stage 1: verify the package ----------------------------------------
$hash = (Get-FileHash $MsiPath -Algorithm SHA256).Hash
if ($hash -ne $ExpectedSha256) { throw "Checksum mismatch: $hash" }
$ev.InstallerSha256 = $hash
$ev.VersionBefore   = Get-AgentVersion

# ---- Stage 2: silent install --------------------------------------------
$log = Join-Path $EvidenceDir "$ChangeRef-$stamp-install.log"
if ($PSCmdlet.ShouldProcess($env:COMPUTERNAME, "Install $MsiPath")) {
  $p = Start-Process msiexec.exe -Wait -PassThru -ArgumentList "/i `"$MsiPath`" /qn /norestart /l*v
`"$log`" $MsiProperties"
  if ($p.ExitCode -notin 0,3010) { throw "msiexec exit $($p.ExitCode). See $log" }
  $ev.InstallExitCode = $p.ExitCode
}

# ---- Stage 3: service identity and startup ------------------------------
if ($ServiceCredential -and $PSCmdlet.ShouldProcess($ServiceName, 'Set logon account')) {
  $svc = Get-CimInstance Win32_Service -Filter "Name='$ServiceName'"
  $r = Invoke-CimMethod -InputObject $svc -MethodName Change -Arguments @{
         StartName = $ServiceCredential.UserName; StartPassword =
$ServiceCredential.GetNetworkCredential().Password }
  if ($r.ReturnValue -ne 0) { throw "Win32_Service.Change returned $($r.ReturnValue)" }
}
if ($PSCmdlet.ShouldProcess($ServiceName, 'Set Automatic + restart-on-failure')) {
  Set-Service -Name $ServiceName -StartupType Automatic
  sc.exe failure $ServiceName reset= 86400 actions= restart/60000/restart/60000// | Out-Null
}

# ---- Stage 4: optional hotfix (service stopped while patching) ----------
if ($HotfixPath -and $PSCmdlet.ShouldProcess($ServiceName, "Apply hotfix $HotfixPath")) {
  $ev.HotfixSha256 = (Get-FileHash $HotfixPath -Algorithm SHA256).Hash
  Stop-Service $ServiceName -Force
  $hlog = Join-Path $EvidenceDir "$ChangeRef-$stamp-hotfix.log"
  $hfArgs = if ($HotfixPath -like '*.msp') { "/p `"$HotfixPath`" /qn /norestart /l*v `"$hlog`"" }
          else { "/i `"$HotfixPath`" /qn /norestart /l*v `"$hlog`"" }
  $h = Start-Process msiexec.exe -Wait -PassThru -ArgumentList $hfArgs
  if ($h.ExitCode -notin 0,3010) { throw "Hotfix exit $($h.ExitCode). See $hlog" }
}

# ---- Stage 5: start and verify ------------------------------------------
Start-Service $ServiceName
Start-Sleep -Seconds 20
$s = Get-CimInstance Win32_Service -Filter "Name='$ServiceName'"
$ev.ServiceState = $s.State; $ev.StartMode = $s.StartMode; $ev.ServiceAccount = $s.StartName
$ev.VersionAfter = Get-AgentVersion
$ev.TimeSync     = (w32tm /query /status | Select-String 'Source|Last Successful').Line
$ev.EndUtc       = (Get-Date).ToUniversalTime()
$ev | ConvertTo-Json -Depth 3 | Out-File (Join-Path $EvidenceDir "$ChangeRef-$stamp-evidence.json")
Write-Host "Done. Now confirm Connections is healthy in the agent and the workstation reports in the web
console." -ForegroundColor Green
```

EXAMPLE · RUNNING THE TEMPLATE

INVOCATION POWERSHELL

```
.\Install-ComplianceAgent.ps1 -MsiPath 'C:\Install\CBAgent\CBAgentSetup.msi' `
  -ExpectedSha256 '<sha256-from-repository>' -ChangeRef 'CHG0012345' -ServiceName
'<AgentServiceName>' `
  -HotfixPath 'C:\Install\CBAgent\<hotfix-package>' -WhatIf
```

Run once with `-WhatIf` to confirm what would change, then without it. The JSON file and both msiexec logs go into the change record.

KEY CONCEPT · WHY RESTART-ON-FAILURE?

If the agent service crashes overnight, events queue (or are missed) until someone notices. `sc.exe failure` tells the Service Control Manager to restart it automatically after 60 seconds, twice, resetting the counter daily. It does not replace monitoring, but it closes the gap between a crash and a human response.

Firewall rule, if the vendor requires inbound traffic

INBOUND RULE SCOPED TO THE SERVER POWERSHELL

```
New-NetFirewallRule -DisplayName 'Compliance Agent (from server)' -Direction Inbound -Protocol TCP `
  -LocalPort <agent-port> -RemoteAddress 10.20.30.40 -Profile Domain -Action Allow
# Outbound is normally allowed by default; if your baseline blocks outbound, mirror the rule with -
Direction Outbound -RemotePort.
```

Scope the rule to the server's address and the Domain profile. On managed estates, deliver it with Group Policy or an Intune firewall rules policy (**Endpoint security › Firewall**) rather than locally, so the rule is documented and cannot drift.

CHAPTER 4

Methods C and D · Remote and Enterprise Deployment

4.1 Method C · Remote installation

Lab IT is often not in the same building as the instrument, and lab PCs are often unreachable by normal RDP. Two sanctioned remote paths are common.

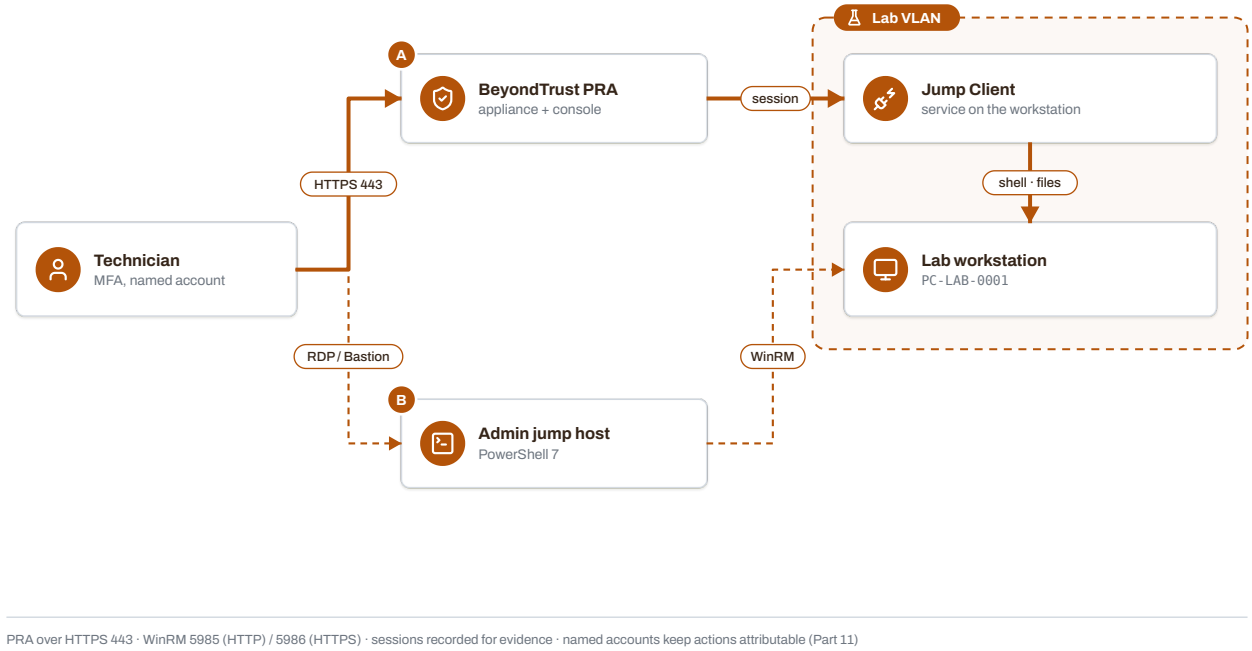


Figure 6. Either path reaches the workstation, but the PRA session also produces a recording you can file with the change as evidence.

METHOD C1 BeyondTrust Privileged Remote Access (PRA)

PRA brokers sessions through an appliance, enforces who may reach which endpoints (Jump Groups and policies), and can record sessions. That recording is useful evidence for a GxP change.

1. In the PRA access console, open the Jump Item for the workstation (a Jump Client installed on it, or a Remote Jump through a Jumpoint). If there is none, send the user a session key or invite from the console so they can start a session.

2. Start the session, sign in with your admin account, and confirm the hostname and OS version.
3. Use the **File Transfer** tab to copy the MSI, hotfix and `Install-ComplianceAgent.ps1` to `C:\Install\CBAgent\`.
4. Either run Method A in the **Screen Sharing** tab, or open the PRA **Command Shell** as administrator and run the Method B script.
5. If the reboot drops the session, wait for the Jump Client to reconnect, or generate a fresh invite and reconnect after the user signs in.

WATCH OUT · ANALYSTS MID-RUN

Do not take over a screen while an instrument run is acquiring data. Confirm with the lab that the instrument is idle; the maintenance window exists for this reason.

METHOD C2 PowerShell remoting (WinRM)

Where WinRM is enabled on the lab VLAN (TCP 5985 HTTP or 5986 HTTPS, both encrypted at the message level in a domain with Kerberos), you can push the files and run the template from a jump host without touching the user's screen.

PUSH AND RUN OVER WINRM — POWERSHELL

```powershell
$pcs  = 'PC-LAB-0001','PC-LAB-0002'
$cred = Get-Credential 'CONTOSO\adm-labit'
$hash = (Get-FileHash .\CBAgentSetup.msi -Algorithm SHA256).Hash
foreach ($pc in $pcs) {
  $s = New-PSSession -ComputerName $pc -Credential $cred
  Invoke-Command -Session $s { New-Item C:\Install\CBAgent -ItemType Directory -Force | Out-Null }
  Copy-Item .\CBAgentSetup.msi, .\Install-ComplianceAgent.ps1 -Destination C:\Install\CBAgent\ -
ToSession $s
  Invoke-Command -Session $s -ScriptBlock {
    & C:\Install\CBAgent\Install-ComplianceAgent.ps1 -MsiPath C:\Install\CBAgent\CBAgentSetup.msi `
      -ExpectedSha256 $using:hash -ChangeRef 'CHG0012345' -ServiceName '<AgentServiceName>'
  }
  Copy-Item 'C:\ProgramData\LabIT\Evidence\*' -Destination ".\evidence\$pc\" -FromSession $s -
Recurse
  Remove-PSSession $s
}
```

NOTE · THE SECOND-HOP PROBLEM

Inside a remote session your credentials cannot be passed on to a file share (Kerberos double hop), so `\\repo\share\CBAgentSetup.msi` will fail with access denied. That is why the example copies files *through* the session with `-ToSession`.

4.2 Method D · Enterprise deployment (ConfigMgr or Intune)

Once the package and commands are proven with Method B, wrap them in your endpoint management platform. The platform handles targeting, maintenance windows, retries and reporting, and its deployment status becomes fleet-wide evidence.

METHOD D1 Configuration Manager application

1. Copy the MSI (and hotfix) to the content share, for example `\\SRV-CM-01\Sources$\Apps\ComplianceAgent\<version>\`.
2. In the console go to **Software Library › Application Management › Applications** and select **Create Application**. Choose *Windows Installer (*.msi file)* and browse to the MSI; ConfigMgr reads the product code automatically.
3. Edit the deployment type's installation program to the silent command line with your properties, for example `msiexec /i "CBAgentSetup.msi" /qn /norestart <SERVERPROPERTY>=...`. The detection method defaults to the MSI product code; keep it.
4. On **User Experience** set *Install for system*, *Whether or not a user is logged on*, and allow the client to handle a 3010 soft reboot.
5. Model the hotfix as a *superseding* application (or a second deployment type) so ConfigMgr upgrades existing installs in order.
6. Distribute content to the distribution points that serve the lab, then **Deploy** to a lab device collection with purpose *Required*. The deployment honours the collection's maintenance window, which is exactly what a validated lab needs.
7. Monitor in **Monitoring › Deployments**; export the success list for the change record.

METHOD D2 Intune Win32 app

1. Put the MSI and a small `install.cmd` (containing the silent command) in a source folder, then wrap it: `IntuneWinAppUtil.exe -c .\src -s install.cmd -o .\out`.
2. In the Intune admin center open **Apps › Windows › + Add** (labelled **+ Create** in newer tenants) and choose *Windows app (Win32)*. Upload the `.intunewin` file.
3. **Program**: install command `install.cmd`; uninstall command `msiexec /x {<ProductCode>} /qn`; install behaviour *System*; device restart behaviour

Determine behavior based on return codes.

4. **Requirements**: 64-bit, minimum OS. **Detection rules**: *MSI* product code (optionally with a version check so the hotfix is detected as a newer version).
5. **Assignments**: *Required* to an Entra ID device group of lab workstations. Use a scope tag so only lab app managers can change it.
6. Use *Supersedence* on the hotfix app to replace the earlier version, and check status under the app's `Device install status`.

WATCH OUT · INTUNE DOES NOT KNOW YOUR LAB SCHEDULE

Intune has no native maintenance window for Win32 apps. For validated lab PCs either use an availability/deadline time in the assignment, deploy as *Available* and install during the window, or keep these devices on ConfigMgr where maintenance windows apply.

NOTE · GROUP POLICY SOFTWARE INSTALLATION

GPO *Software installation* (assigning an MSI to computers) still works but offers no reporting, no maintenance windows and no supersedence. Treat it as a last resort for small, unmanaged labs.

CHAPTER 5

Hotfix, Verification and Evidence

5.1 The hotfix lifecycle

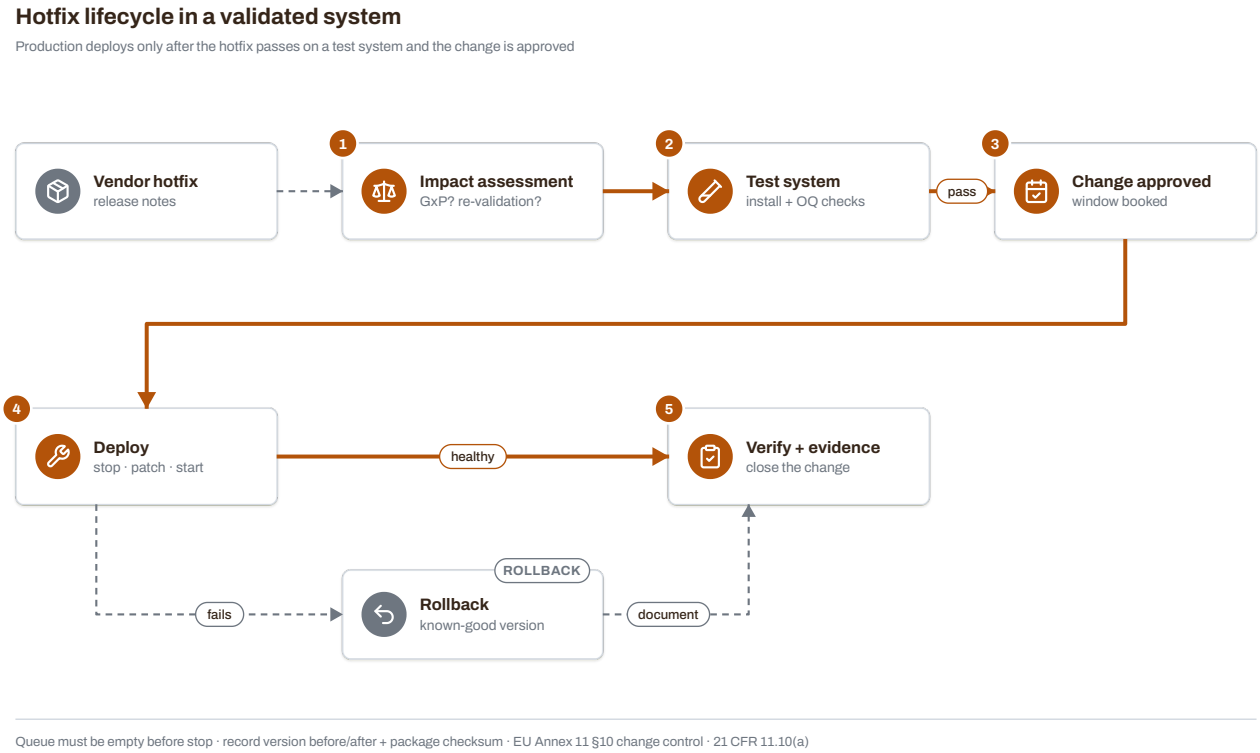


Figure 7. A hotfix touches validated software, so it reaches production only after a test-system pass and an approved change, with rollback planned before you start.

Impact assessment

Read the vendor release notes with Quality or the CSV (computer system validation) lead. Record what the hotfix fixes, whether it touches data capture, signatures or audit trail, whether server and agent versions must match, and whether existing OQ test scripts cover the change. The answer decides how much re-testing is needed.

Deployment steps (any method)

1. Confirm the approved hotfix package and its checksum from the controlled repository.
2. Confirm the agent queue is empty (all events uploaded) so nothing is in flight. In the agent window check that no unsent items remain; in the console confirm the last contact time is current.

3. Stop the agent service if the vendor notes require it: `Stop-Service <AgentServiceName>`.
4. Run the hotfix installer, or follow the vendor's file-copy steps exactly as written.
5. Start the service, verify **Connections** in the agent and visibility in the web console.
6. Confirm the version fields (agent **About**, Programs list, or `Get-AgentVersion` in the template) reflect the hotfix.
7. If the workstation is in a test batch, run a sample workflow: create a test file in a monitored folder, see the event arrive, and sign it off with a test account.

5.2 Verification checklist

Workstation

- ☐ Agent service *Running*, startup *Automatic*, recovery actions set
- ☐ Connections tab healthy
- ☐ Agent and hotfix versions as approved
- ☐ Time synchronized with the domain source
- ☐ No installer errors in the Application event log (source MsiInstaller)

Server / console

- ☐ Workstation listed under Workstations
- ☐ Associated with the correct System and lab
- ☐ Last-contact time current
- ☐ Test event received and signed (if in test batch)
- ☐ Stakeholders notified that the PC is ready for test scripts

QUICK VERIFICATION — POWERSHELL

```
Get-CimInstance Win32_Service -Filter "Name='<AgentServiceName>'" | Select Name, State, StartMode,
StartName
Get-WinEvent -FilterHashtable @{LogName='Application'; ProviderName='MsiInstaller'; StartTime=(Get-
Date).AddHours(-2)} |
  Select TimeCreated, Id, Message | Format-Table -Wrap
w32tm /query /status
```

5.3 Evidence to capture

Item	Supports (ALCOA+)
Change ID and maintenance window	Contemporaneous, controlled change
Agent version before and after; hotfix ID and checksum	Accurate, Original (the approved bits were installed)
Screenshots of agent Connections and console Workstations	Complete, Available

Hostname and OS details	Attributable to the right system
Operator identity, date/time (UTC), initials on paper forms	Attributable, Contemporaneous
msiexec logs and JSON evidence file; PRA session recording	Enduring, Consistent

CHAPTER 6

Rollback and Troubleshooting

6.1 Rollback

1. Stop the agent service.
2. Revert or uninstall the hotfix per vendor guidance (for an MSP patch: `msiexec /package {<ProductCode>} MSIPATCHREMOVE={<PatchCode>} /qn`, only if the vendor marked it uninstallable).
3. If needed, uninstall the agent (`msiexec /x {<ProductCode>} /qn /norestart`), reboot, and install the known-good version.
4. Re-add the connection in the Agent Configuration Utility and validate as in Chapter 5.
5. Document the rollback and its reason in the change record; open a problem record if the vendor needs to investigate.

6.2 Troubleshooting flow

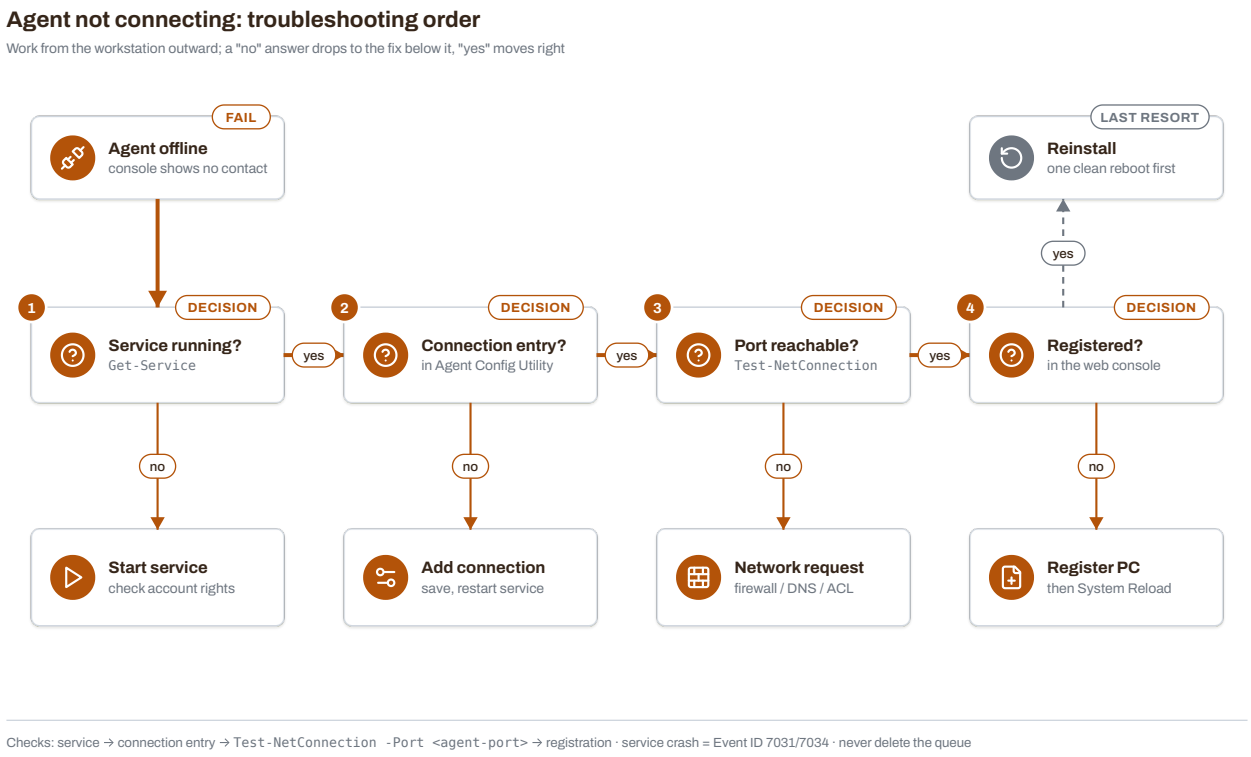


Figure 8. Check what you control on the PC first, then the network, then the server; reinstalling is the last resort, never the first move.

Symptom	Likely cause and fix
Agent installed but never connects	No connection entry. Open the Agent Configuration Utility and add it; save; restart the service.
Connection entry exists, still red	Workstation not registered or server not reloaded. Add it under Workstations and perform a System Reload.
`TcpTestSucceeded : False`	Firewall or VLAN ACL. Raise a request with the network team listing source, destination and `<agent-port>`.
Service starts then stops (Event ID 7034/7031)	Service account password expired or lacks *Log on as a service*; or corrupt config. Check the System log and the vendor log folder.
Events have wrong timestamps	Time service not synchronized. `w32tm /resync`, then check the domain hierarchy.
msiexec returns 1603	Generic fatal error. Search the verbose log for "Return value 3"; common causes are a pending reboot or files locked by a running agent.
Remote session lost after reboot	Generate a fresh PRA invite and reconnect after sign-in, or wait for the Jump Client.
Cannot sign in to a Windows 11 PC again after reboot	Request a targeted reboot of that workstation and retry; check that cached credentials or the secure channel are intact.

DANGER · DO NOT "FIX" BY DELETING THE QUEUE

Deleting an agent's local queue or database to clear an error destroys unsent audit-trail events. That is a data-integrity incident. Escalate to the system owner and vendor instead.

CHAPTER 7

Summary, Glossary and Review

7.1 Chapter summary

- A compliance agent records events at the instrument PC and sends them to a server that keeps the audit trail; registration ties the workstation to the right System.
- Plan the service identity, port and firewall path before the window; confirm time sync.
- Install interactively for a pilot, silently with msiexec for repeatability, remotely through PRA or WinRM, and at scale through ConfigMgr or Intune.
- Hotfixes are changes to a validated system: assess, test, approve, deploy, verify, document.
- Evidence (versions, checksums, logs, screenshots, operator, time) is part of the job, not an afterthought.

7.2 Glossary

GxP	Umbrella term for "good practice" regulations (GMP, GLP, GCP) governing regulated product work.
21 CFR Part 11	US FDA rule on electronic records and electronic signatures.
EU Annex 11	EU GMP guidance on computerised systems.
ALCOA+	Data-integrity principles: Attributable, Legible, Contemporaneous, Original, Accurate, plus Complete, Consistent, Enduring, Available.
CSV	Computer system validation; documented evidence that a system does what it is intended to do.
IQ / OQ	Installation and operational qualification test phases.
gMSA	Group managed service account; AD account whose password Windows rotates automatically.
Jump Client	BeyondTrust PRA agent that lets authorised users start sessions to an endpoint.
Win32 app	Intune app type for packaged installers (.intunewin) with detection rules.
Supersedence	ConfigMgr/Intune relationship that replaces an older app version with a newer one.

Check your understanding

1. Why is the workstation registered in the web console before the agent is installed?
2. The agent service runs as LocalSystem but must monitor a folder on a file share. What identity would you choose instead, and why?
3. Your WinRM script fails to read the MSI from `\\repo\share`. What is the cause and how does the template avoid it?
4. Name four evidence items for a hotfix change and the ALCOA+ principle each supports.
5. Why might you keep validated lab PCs on ConfigMgr rather than Intune for hotfix deployment?